

Small Report on Robust KalmaNet Trust for QCar

Quang Huy Nguyen

June 4, 2026

Abstract

This report summarizes the Robust KalmaNet estimator implemented for the QCar platform. The estimator keeps the physical QCar prediction model and learns a trust mechanism for the measurement update. The trust mechanism is represented by a learned measurement mask and a learned Kalman-style gain. Its purpose is to reduce the influence of corrupted GPS position, heading, and velocity measurements while preserving normal Extended Kalman Filter (EKF)-like behavior on clean data.

1 Motivation

The QCar observer receives measurements from GPS, IMU, steering, wheel-speed, and motor tachometer signals. A classical EKF can produce good estimates when the measurement noise model is correct, but it can also over-trust a sensor during abnormal events such as GPS jumps, GPS dropout, frozen velocity, wheel speed scaling, or noisy inertial data.

The Robust KalmaNet design addresses this issue by separating the estimation problem into two parts:

1. a physics-based prediction step that propagates the vehicle state using the QCar kinematic model;
2. a learned update step that decides how much to trust each measurement channel before applying a Kalman-style correction.

The key idea is not to replace the vehicle model with a black-box neural network. Instead, learning is concentrated on the part where uncertainty and sensor faults are most difficult to model manually: measurement trust.

2 State, Measurements, and Raw Inputs

The active training surface uses the four-state vector

$$\mathbf{x}_k = [x_k \ y_k \ \psi_k \ v_k]^T, \quad (1)$$

where x and y are global position, ψ is heading, and v is forward velocity. Some recorded files contain an additional yaw-rate channel, but the training data builder slices the target and measurement arrays to the first four channels.

The measurement vector is

$$\mathbf{z}_k = [x_{gps,k} \ y_{gps,k} \ \psi_{fused,k} \ v_{tach,k}]^T, \quad \mathbf{z}_k = H\mathbf{x}_k + \mathbf{n}_k, \quad H \approx I_4. \quad (2)$$

The raw branch inputs used by the model are grouped as:

- IMU branch: acceleration and yaw-rate channels;

- steering branch: steering angle and related motion context;
- wheel branch: four wheel-speed channels;
- availability metadata: GPS valid flag, heading valid flag, GPS hold flag, and GPS age.

3 Prediction Model

The current configuration uses `predictor_mode: kinematic`, so the prediction step is analytical and has no learned parameters. The vehicle is propagated using a bicycle-model yaw-rate approximation:

$$\dot{\psi}_k = \frac{v_k \tan(\delta_k)}{L}, \quad (3)$$

where δ_k is the steering angle and L is the wheelbase. The prior state is

$$x_k^- = x_{k-1}^+ + v_k \cos(\psi_{k-1}^+) \Delta t, \quad (4)$$

$$y_k^- = y_{k-1}^+ + v_k \sin(\psi_{k-1}^+) \Delta t, \quad (5)$$

$$\psi_k^- = \text{wrap} \left(\psi_{k-1}^+ + \frac{v_k \tan(\delta_k)}{L} \Delta t \right). \quad (6)$$

The active longitudinal option is `velocity_lag`. In this mode, velocity is predicted from throttle using

$$\dot{v}_k = \frac{-v_{k-1}^+ + g u_k}{\tau}, \quad v_k^- = \text{clip} (v_{k-1}^+ + \dot{v}_k \Delta t). \quad (7)$$

The current config uses $L = 0.2$, $\tau = 0.301$, gain $g = 6.598$, maximum velocity 2.0 m/s, and maximum acceleration 2.0 m/s².

4 Learned Trust Update

The learned update receives the prior state $\mathbf{x}_k^-$, the previous updated state $\mathbf{x}_{k-1}^+$, the current measurement $\mathbf{z}_k$, the previous measurement $\mathbf{z}_{k-1}$, and sensor availability metadata.

The innovation and measurement difference are

$$\mathbf{r}_k = \text{wrap}(\mathbf{z}_k - H \mathbf{x}_k^-), \quad (8)$$

$$\Delta \mathbf{z}_k = \text{wrap}(\mathbf{z}_k - \mathbf{z}_{k-1}). \quad (9)$$

The hard availability vector is

$$\mathbf{a}_k = [g_k \quad g_k \quad h_k \quad 1]^T, \quad (10)$$

where g_k is the GPS-position availability flag and h_k is the heading availability flag. The update feature vector combines normalized state increment, innovation, measurement increment, hard availability, GPS hold flag, GPS age, and innovation magnitudes. For the four-state model this produces a 26-dimensional update feature vector.

The trust mask is generated by a small MLP:

$$\mathbf{m}_k = \sigma(\text{MLP}(\mathbf{f}_k)). \quad (11)$$

The measurement part of this vector, $\mathbf{m}_k^z$, is interpreted as learned measurement trust. Clean channels should have values near one. Attacked or unreliable channels should move toward zero.

The Kalman-style gain is generated by a GRU and an MLP:

$$\mathbf{h}_k = \text{GRU}(\mathbf{m}_k \odot \mathbf{f}_k, \mathbf{h}_{k-1}), \quad (12)$$

$$K_k = s \tanh(\text{MLP}(\mathbf{h}_k)). \quad (13)$$

The current tanh scale is $s = 1.2$. Both the measurement mask and gain are smoothed with exponential moving averages using $\alpha = 0.35$.

The final robust correction is

$$\mathbf{x}_k^+ = \mathbf{x}_k^- + K_k (\mathbf{a}_k \odot \mathbf{m}_k^z \odot \mathbf{r}_k). \quad (14)$$

This equation is the main trust mechanism. A large raw innovation from a bad GPS or velocity measurement cannot directly force a large state jump if the learned mask suppresses that channel. The implementation also limits per-step update corrections with the configured clamp

$$[0.35, 0.35, 0.35, 0.80]$$

for $[x, y, \psi, v]$.

5 Dataset and Training Procedure

The dataset recorder saves synchronized samples into compressed `.npz` files and matching JSON metadata. The active training configuration points to `robust_kalmanet_dataset_V0_20260417_094048.npz`. Its metadata reports 52,921 samples, 671.84 seconds of driving, and a mean timestep of 0.0127 seconds.

The trainer uses sliding windows with length $T = 50$ for updater training and $T = 80$ in Phase C fine tuning. The validation split is the final 20 percent of the sequence. Measurements are rebuilt offline using the local QCar EKF heading path, and invalid GPS x/y samples use the configured `freeze` behavior.

Because the predictor is kinematic, Phase A is automatically skipped. Training therefore proceeds as:

- Phase B: freeze the predictor and train the learned updater;
- Phase C: fine tune the full runtime loop with teacher forcing disabled.

The state loss uses weighted MSE with weights

$$W = [1.0, 1.0, 2.5, 1.2].$$

The combined objective is

$$\mathcal{L} = \lambda_u L_{upd} + \lambda_p L_{pred} + \lambda_m L_{mask} + \lambda_K L_K + \lambda_{\Delta K} L_{\Delta K} + \lambda_{\Delta m} L_{\Delta m}. \quad (15)$$

Here L_{upd} trains the final corrected state, L_{pred} keeps the prior reasonable, L_{mask} supervises measurement trust under synthetic attacks, L_K penalizes large gains, and the temporal terms reduce gain and mask chatter. Current Phase C weights are $\lambda_u = 0.85$ and $\lambda_p = 0.15$.

6 Sensor Attack Augmentation

Robustness is trained by corrupting raw branches and measurement channels. The active augmentation mode is `mixed`. It includes:

- raw branch attacks on IMU, steering, and wheel-speed branches;
- direct measurement attacks on heading and velocity;
- GPS position attacks on x/y .

The enabled raw attack types are bias, scale, freeze, noise, ramp, and zero-out. GPS-specific attacks include noise, freeze, jump, dropout, and reacquisition offsets. The active probabilities are:

Attack group	Probability
Raw branch attack	0.25
Direct measurement attack	0.35
GPS position attack	0.45

Only one raw branch is attacked at a time. The supervision target for the measurement trust mask is one for clean measurement channels and zero for attacked channels.

7 Observed Training Result

The saved training history in `models/robust_kalmanet.train_history.json` contains 20 logged epochs after Phase A was skipped. The best robust checkpoint was selected at epoch 15 in Phase B.

Checkpoint point	Clean validation	Attacked validation	Selection loss
Epoch 1	0.00668	1.10951	0.66838
Best robust, epoch 15	0.01751	0.40279	0.24868
Final Phase C, epoch 20	0.02408	0.43457	0.27037

The clean validation loss increases after aggressive attack training, but the attacked validation loss decreases strongly. This is the expected tradeoff for a robust-trust estimator: the model becomes more conservative on measurements so that it can reject corrupted sensor channels.

8 Conclusion

The Robust KalmaNet trust mechanism for QCar combines a deterministic kinematic prediction model with a learned measurement trust mask and a learned Kalman-style gain. The predictor preserves vehicle physics, while the updater learns when GPS, heading, and velocity should be trusted. The saved training run shows that attack-aware training substantially reduces attacked validation loss, which supports the intended use case: stable QCar state estimation under sensor faults and GPS disturbances.